

# Django Asoslari — 1-Dars Qo'llanmasi va Ish Rejasi

Boshlang'ich guruh uchun · O'rnatishdan runserver'gacha to'liq amaliy dars

|                  |                                                                                                                                                                 |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mavzu            | Django framework bilan tanishuv — o'rnatish, virtual muhit, birinchi app                                                                                        |
| Davomiyligi      | ~120 daqiqa (2 akademik soat)                                                                                                                                   |
| Daraja           | Boshlang'ich (Python asoslarini biladigan guruh uchun)                                                                                                          |
| Kerakli anjomlar | Noutbuk/kompyuter, Python 3.10+, kod muharriri (VS Code tavsiya etiladi), internet                                                                              |
| Dars maqsadi     | Har bir o'quvchi darsdan keyin o'z Django loyihasini yaratib, HttpResponse orqali HTML sahifalarni brauzerda ishga tushira olishi kerak (Model va Template'siz) |

## 1. Dars rejasi (vaqt bo'yicha)

**Diqqat:** bu birinchi dars uchun mo'ljallangan — Model va migratsiya bu darsda ISHLATILMAYDI. Maqsad: o'quvchilar Django loyihasini ishga tushirib, HttpResponse orqali to'g'ridan-to'g'ri HTML kod qaytarishni o'rganishi. Model/ma'lumotlar bazasi — keyingi darsning mavzusi.

| Vaqt   | Mavzu                            | Faoliyat                                                                                                                                        |
|--------|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| 5 daq  | Kirish va motivatsiya            | Django'da yaratilgan real saytlar (Instagram, Spotify) misolida qiziqish uyg'otish                                                              |
| 10 daq | Django nima? MVT arxitekturası   | Model - View - Template tushunchalarini umumiy tanishtirish (chuqur emas, faqat View qismiga urg'u)                                             |
| 5 daq  | Python va pip tekshirish         | <code>python --version</code> , <code>pip --version</code> buyruqlari                                                                           |
| 10 daq | Virtual muhit yaratish           | <code>venv</code> nima uchun kerakligini tushuntirish + yaratish/faollashtirish                                                                 |
| 5 daq  | Django o'rnatish                 | <code>pip install django</code>                                                                                                                 |
| 10 daq | Loyiha (project) yaratish        | <code>django-admin startproject</code> , papka tuzilishini ko'rsatish                                                                           |
| 10 daq | Ilova (app) yaratish             | <code>python manage.py startapp</code> , app nima uchun alohida bo'ladi                                                                         |
| 5 daq  | <code>settings.py</code> sozlash | <code>INSTALLED_APPS</code> ro'yxatiga yangi appni qo'shish                                                                                     |
| 10 daq | View yaratish — oddiy matn       | <code>views.py</code> da HttpResponse bilan oddiy matn qaytarish                                                                                |
| 15 daq | View yaratish — HTML kod bilan   | HttpResponse ichiga to'g'ridan-to'g'ri HTML teglar ( <code>h1</code> , <code>p</code> , <code>div</code> , <code>style</code> ) yozib qaytarish |
| 10 daq | URL sozlash                      | <code>urls.py</code> da bir nechta sahifa uchun yo'nalish (routing) qo'shish                                                                    |
| 10 daq | Serverni ishga tushirish         | <code>runserver</code> , brauzerda natijani ko'rish, xatolarni birga tuzatish                                                                   |
| 15 daq | Amaliy mashq                     | Har bir o'quvchi mustaqil 2-3 ta yangi HTML sahifa (view) yaratadi, o'qituvchi aylanib tekshiradi                                               |
| 5 daq  | Yakunlash va savol-javob         | Kunning asosiy tushunchalarini qisqacha takrorlash                                                                                              |
| 5 daq  | Uyga vazifa berish               | Vazifa shartlarini tushuntirish (pastda batafsil)                                                                                               |

## 2. Batafsil amaliy qo'llanma (o'quvchilar bilan birga bajariladi)

### 1 Python va pip borligini tekshirish

#### NIMA UCHUN KERAK?

**Python** — Django yozilgan dasturlash tili; u kompyuteringizda o'rnatilmasa, Django ham ishlamaydi. **pip** — Python uchun paket (kutubxona) menejeri: boshqa dasturchilar yozgan tayyor kod bo'laklarini (masalan, Django'ning o'zini) internetdan yuklab, loyihangizga qo'shib beradi. Har qanday Python loyihasini boshlashdan oldin, ikkalasi ham to'g'ri o'rnatilganini tekshirib olish kerak — aks holda keyingi barcha buyruqlar ishlamay qoladi.

#### AMALIYOT — TERMINALGA YOZING

```
python --version
pip --version
```

Ikkalasi ham versiya raqamini chiqarsa (masalan Python 3.11.4) — hammasi tayyor, davom etamiz.

### 2 Virtual muhit (venv) yaratish va faollashtirish

#### NIMA UCHUN KERAK?

Har bir Django loyihasi o'ziga xos kutubxonalar to'plamiga (va ularning aniq versiyalariga) muhtoj bo'ladi. Agar hammasi kompyuterning "umumiy" joyiga o'rnatilsa, bitta loyiha uchun Django 5.0, boshqasi uchun Django 4.2 kerak bo'lganda ziddiyat chiqadi. **Virtual muhit (venv)** — har bir loyiha uchun alohida, izolyatsiyalangan "quti" yaratadi: shu quti ichiga o'rnatilgan kutubxonalar faqat shu loyihaga tegishli bo'ladi va boshqa loyihalarga umuman ta'sir qilmaydi.

#### WINDOWS UCHUN

```
python -m venv env
env\Scripts\activate
```

#### LINUX / MACOS UCHUN

```
python3 -m venv env
source env/bin/activate
```

**Qanday bilaman faollashganini?** Terminal qatorining boshida (env) degan yozuv paydo bo'ladi. Shu yozuv bo'lmasa, keyingi buyruqlar (masalan Django o'rnatish) noto'g'ri joyga tushib qolishi mumkin.

### 3 Django'ni o'rnatish

#### NIMA UCHUN KERAK?

**Django** — Python ustiga qurilgan, veb-saytlar yaratishni tezlashtiradigan tayyor "freymvork" (framework). U bo'lmasa, har bir sayt uchun serverni qabul qilish, so'rovlarni qayta ishlash, ma'lumotlar bazasi bilan ishlash kabi narsalarni noldan o'zimiz yozishimiz kerak bo'lardi — bu haftalab vaqt oladi. Django buning barchasini tayyor beradi, bizga faqat o'z loyihamizning mantig'ini yozish qoladi. `pip install django` buyrug'i Django kodini internetdan yuklab, hozirgi faol venv ichiga o'rnatadi.

#### AMALIYOT — TERMINALGA YOZING

```
pip install django
```

### 4 Yangi loyiha (project) yaratish

#### NIMA UCHUN KERAK?

**django-admin** — Django o'rnatilganda avtomatik qo'shiladigan buyruq qurol (bu Python fayl emas, balki terminal buyrug'i). **startproject** — unga "yangi loyiha uchun barcha boshlang'ich fayl va papkalarni yaratib ber" deb buyuruvchi qism. Natijada `settings.py` (loyihaning barcha sozlamalari), `urls.py` (manzillar ro'yxati) va `manage.py` (loyihani boshqarish uchun asosiy fayl) kabi fayllar avtomatik yaratiladi. Biz loyihaga `config` deb nom berdik — bu ixtiyoriy, istalgan nom qo'yish mumkin, lekin "config" (sozlamalar) nomi keng tarqalgan va tushunarli.

#### AMALIYOT — TERMINALGA YOZING

```
django-admin startproject config .
```

Oxiridagi nuqta (.) — "loyihani joriy papkaning ichiga yarat, qo'shimcha ichki papka ochma" degani. Nuqta qo'yilmasa, config nomli qo'shimcha tashqi papka ham yaratiladi, bu boshlang'ichlar uchun ortiqcha chalkashlik tug'diradi.

## 5 Ilova (app) yaratish

### NIMA UCHUN KERAK?

Django'da **loyiha (project)** va **ilova (app)** tushunchalari farq qiladi: loyiha — butun sayt, app esa uning bitta mustaqil qismi (masalan: *blog*, *users*, *shop*). Bitta loyiha ichida bir nechta app bo'lishi mumkin — bu kodni tartibli, alohida bo'laklarga bo'lib saqlash imkonini beradi. `manage.py` — startproject yaratgan boshqaruv fayli; unga startapp blog deb buyursak, u blog nomli yangi papka va uning ichiga `views.py`, `models.py` kabi tayyor fayl-qoliplarni yaratib beradi.

### AMALIYOT — TERMINALGA YOZING

```
python manage.py startapp blog
```

## 6 Appni settings.py ga qo'shish

### NIMA UCHUN KERAK?

App papkasini yaratish uni Django'ga "tanishtirmaydi" — Django hali bu appning borligini bilmaydi. `settings.py` — butun loyihaning "markaziy boshqaruv paneli": qaysi applar faol, qaysi baza ishlatiladi, til va vaqt zonasi kabi hamma narsa shu yerda belgilanadi. `INSTALLED_APPS` — faol applar ro'yxati. Agar 'blog' ni shu ro'yxatga qo'shmasak, Django uning ichidagi model, template yoki boshqa imkoniyatlarni umuman ko'rmaydi va ishlatolmaydi — shuning uchun bu qadam hech qachon o'tkazib yuborilmaydi.

### CONFIG/SETTINGS.PY FAYLIDA

```
config/settings.py

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'blog', # <-- yangi appimiz shu yerga qo'shildi
]
```

## 7 Birinchi view — oddiy matn qaytarish

### NIMA UCHUN KERAK?

Django'ning ishlash mantig'i (MVT — Model, View, Template) shunday: foydalanuvchi brauzerda biror manzilni ochganda, so'rov (**request**) serverga keladi va Django uni ma'lum bir Python funksiyaga — **view**'ga — yo'naltiradi. View'ning vazifasi — shu so'rovga qanday javob (**response**) qaytarishni hal qilish. Eng oddiy javob turi — `HttpResponse`: unga nima bersak, brauzerga aynan shu qaytadi.

### AMALIYOT — TERMINALGA YOZING

```
blog/views.py

from django.http import HttpResponse

def home(request):
    return HttpResponse("Salom, Django dunyosiga xush kelibsiz!")
```

request — har bir view funksiyasi majburiy qabul qiladigan parametr; unda foydalanuvchi so'rovi haqidagi barcha ma'lumot (qaysi manzil so'ralgani, qanday brauzerdan, qanday ma'lumot yuborilgani) saqlanadi.

## 8 HttpResponse ichiga HTML kod yozish

### NIMA UCHUN KERAK?

`HttpResponse` ichiga oddiy matn o'rniga **HTML teglarini** yozib yuborsak, brauzer buni matn sifatida emas, balki formatlangan sahifa sifatida ko'rsatadi — chunki brauzer har doim kelgan javobni HTML deb o'qishga tayyor turadi. Bu usul Django Template fayllarisiz ham sahifa qanday chiqishini tez his qilish uchun juda yaxshi boshlang'ich mashq, garchi katta loyihalarda HTML odatda alohida `.html` fayllarda saqlansa ham.

blog/views.py

```

from django.http import HttpResponse

def home(request):
    html = """
    <html>
        <head><title>Bosh sahifa</title></head>
        <body style="font-family: Arial; text-align: center; margin-top: 50px;">
            <h1 style="color: #0369a1;">Salom, Django dunyosiga xush kelibsiz!</h1>
            <p>Bu sahifa hech qanday shablon (template) faylisiz, to'g'ridan-to'g'ri Python kod ichida yozilgan.</p>
        </body>
    </html>
    """
    return HttpResponse(html)

def about(request):
    html = """
    <html>
        <body style="font-family: Arial; text-align: center; margin-top: 50px;">
            <h1>Biz haqimizda</h1>
            <p>Bu — Django'da yaratilgan ikkinchi sahifamiz.</p>
        </body>
    </html>
    """
    return HttpResponse(html)

```

**Muhim izoh:** bu usul faqat o'rganish uchun qulay. Katta loyihalarda HTML kodni Python ichida yozish o'rniga alohida .html shablon fayllar va Django Template tizimidan foydalaniladi — bu keyingi darslarda o'rgatiladi.

## 9 URL manzillarini sozlash

### NIMA UCHUN KERAK?

View funksiyasini yozgan bo'lsak ham, Django hali "qaysi manzilga kirilganda qaysi view chaqirilishi kerak"ligini bilmaydi. Buni **urls.py** fayli hal qiladi — u manzillar (URL) bilan view funksiyalarini bir-biriga bog'lovchi "yo'l xaritasi". Har bir app o'z ichida kichik urls.py'ga ega bo'lishi, so'ng bu kichik xarita asosiy loyihaning (config) urls.py fayliga include() orqali ulanishi — Django'da tavsiya etiladigan tartib, chunki loyiha kattalashganda har bir app o'z manzillarini mustaqil boshqarib turadi.

### APP ICHIDA YANGI FAYL

blog/urls.py (yangi fayl)

```

from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
    path('about/', views.about, name='about'),
]

```

### ASOSIY LOYIHAGA ULASH

config/urls.py

```

from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('blog.urls')),
]

```

## 10 Serverni ishga tushirish

### NIMA UCHUN KERAK?

manage.py — loyihani boshqarish uchun Django avtomatik yaratgan asosiy fayl (migratsiya, app yaratish kabi barcha buyruqlar shu fayl orqali beriladi). runserver buyrug'i esa kompyuterda vaqtinchalik, faqat o'zimiz ko'rish uchun mo'ljallangan "rivojlantiruvchi server"ni ishga tushiradi — u odatda 127.0.0.1:8000 manzilida (faqat shu kompyuterda) ishlaydi. Bu server production (haqiqiy, foydalanuvchilar kiradigan) sayt uchun

emas, faqat kod yozish jarayonida natijani tekshirish uchun ishlatiladi.

## AMALIYOT — TERMINALGA YOZING

```
python manage.py runserver
```

Brauzerda oching: **http://127.0.0.1:8000/** — rangli "Salom, Django dunyosiga xush kelibsiz!" sahifasini, **http://127.0.0.1:8000/about/** manzilida esa ikkinchi sahifani ko'rasiz.

### 3. Ko'p uchraydigan xatolar va yechimlari

#### **ModuleNotFoundError: No module named 'django'**

Sabab: virtual muhit faollashtirilmagan yoki Django o'sha muhitga o'rnatilmagan. Yechim: `(env)` yozuvi borligini tekshiring, keyin `pip install django` qayta bajaring.

#### **Error: That port is already in use**

Sabab: 8000-port band. Yechim: `python manage.py runserver 8001` kabi boshqa port bering.

#### **NameError yoki IndentationError HTML kod ichida**

Sabab: uch qo'shtirnoq (`"""`) bilan ochilgan matn to'g'ri yopilgan yoki qatorlar noto'g'ri chekinishga (indentation) ega. Yechim: HTML matnining boshi va oxiridagi `"""` larni diqqat bilan tekshiring.

#### **Page not found (404)**

Sabab: `urls.py` fayliga yangi sahifa (masalan `about/`) qo'shilmagan yoki app'ning `urls.py` asosiy `config/urls.py` ga ulanmagan. Yechim: ikkala `urls.py` faylini ham tekshiring.

**O'qituvchi uchun maslahat:** kodni ekranda yozayotganda sekin yozing va har qatordan keyin nima uchun shunday yozganingizni ovoz chiqarib tushuntiring. O'quvchilarga kodni ko'chirib olishga emas, birga qo'lda terishga undang — bu ularning ham "qo'l xotirasini" mustahkamlaydi.

### 4. Uyga vazifa

- Yana 2 ta yangi view yarating: `contact` (bog'lanish ma'lumotlari — telefon, email) va `skills` (o'zingiz bilgan texnologiyalar ro'yxati, masalan `<ul><li>` teglari bilan).
- Har bir view ichidagi HTML kodda kamida bitta `<h1>`, bitta `<p>` va inline style (rang yoki shrift) ishlatilgan bo'lishi kerak.
- Har bir yangi view uchun `urls.py` ga mos manzil qo'shing (masalan `/contact/`, `/skills/`).
- Serverni ishga tushirib, barcha sahifalarni (`/`, `/about`, `/contact`, `/skills`) skrinshot qilib yuboring.